

TP5 Java Yassine Hamrouni

Exercice 1 - Manifeste d'un porte conteneur

1- Écrire le type Container

```
public record Container(String destination, int weight){
    public Container{
        Objects.requireNonNull(destination, "Destination required");
        if(weight < 0){
            throw new IllegalArgumentException("Weight must be positive.");
        }
    }
}
```

2- Écrire le type Manifest

```
public class Manifest{
    private final LinkedList<Container> containerList;

    public Manifest(){
        this.containerList = new LinkedList<Container>();
    }

    public void add(Container c){
        Objects.requireNonNull(c, "Container must be not null");
        this.containerList.add(c);
    }
}
```

3- On souhaite maintenant pouvoir afficher un manifeste

```
@Override
public String toString(){
    int i=1;
    var sb = new StringBuilder();
    for(var container: containerList){
        sb.append(i).append(" ").append(container.toString()).append("\n");
        i++;
    }

    return sb.toString();
}
```

4 Un porte-conteneur, comme son nom ne l'indique pas, peut aussi transporter des passagers. Un Passenger est défini par une destination uniquement, les passagers ne sont pas assez lourds pour avoir un vrai poids.

Dans un premier temps, définir un Passenger afin que l'on puisse créer un passager uniquement avec sa destination. Puis expliquer comment modifier Manifest pour que l'on puisse enregistrer aussi bien des conteneurs que des passagers.

Pour l'affichage, un passager affiche la destination ainsi que "(passenger)" entre parenthèse (cf le code plus bas).

Écrire le code de Passenger et modifier le code de Manifest de telle façon que le code ci-dessous fonctionne.

Interface:

```
public interface Transportable{
    @Override
    String toString();
}
```

Passenger:

```
public record Passenger(String destination) implements Transportable{
    public Passenger{
        Objects.requireNonNull(destination, "Destination required");
    }

    @Override
    public String toString(){
        return destination+" [passenger]";
    }
}
```

Container:

```
@Override
public String toString(){
    return this.destination+" "+this.weight+" kg";
}
```

Manifest:

```
@Override
public String toString(){
    int i=0;
    var sb = new StringBuilder();
```

```

    for(var transport: TransportableList){
        sb.append(i).append(" ").append(transport.toString()).append("\n");
        i++;
    }
    return sb.toString();
}

```

5- On souhaite ajouter une méthode price à Manifest qui calcule le prix pour qu'un conteneur ou qu'un passager soit sur le bateau.

Interface:

```

public interface Transportable{
    @Override
    String toString();
    int price();
}

```

Passenger:

```

@Override
public int price(){
    return 10;
}

```

Container:

```

@Override
public int price(){
    return this.weight*2;
}

```

Manifest:

```

public int price(){
    int sum=0;
    for(var transported: TransportableList){
        sum += transported.price();
    }
    return sum;
}

```

6- On veut maintenant rajouter une méthode weight à Manifest qui renvoie le poids total en considérant qu'un passager n'a pas de poids.

```

public int weight(){
    int sum=0;
    for(var transported: TransportableList){

```

```

        if(transported instanceof Container){
            Container container = (Container) transported;
            sum += container.weight();
        }
    }
    return sum;
}

```

7- on introduit une méthode `removeAllContainersFrom(destination)` qui supprime tous les conteneurs liés à une destination. S'il n'y a pas de conteneur pour cette destination, on ne fait rien.

```

public void removeAllContainersFrom(String dest){
    LinkedList<Transportable> toRemove = new LinkedList<>();
    for(var transported: TransportableList){
        if(transported instanceof Container){
            Container container = (Container) transported;
            if(container.destination().equals(dest)){
                toRemove.add(transported);
            }
        }
    }
    TransportableList.removeAll(toRemove);
}

```

8- Pour résoudre la question précédente, au lieu de faire des appels de méthode, on peut aussi utiliser `instanceof`. Expliquer comment on peut utiliser `instanceof`. Puis expliquer, selon vous, quel est le problème d'utiliser `instanceof` dans ce contexte et pourquoi on ne doit pas l'utiliser.

Utiliser `instanceof` nécessite des informations sur les types concrets d'objets, ce qui peut aller à l'encontre de l'encapsulation et de l'abstraction. L'idée de l'orienté objet est de se concentrer sur ce que fait un objet (comportement), pas ce qu'il est (son type).